CV TECHNOLOGY INTERVIEW

Tài liệu ôn phỏng vấn công nghệ dựa trên CV Backend Intern/Junior

---

Phạm vi CV: CollabBoard — Distributed Real-Time Collaborative Whiteboard; RushSeat — High-Concurrency Flash-Sale & Seat Reservation System.

Công nghệ được nêu: Java 21, Spring Boot, WebSocket/STOMP, Spring Security/JWT, Redis, Kafka, PostgreSQL, JPA/Hibernate, Docker, AWS EC2/RDS, GitHub Actions, JUnit, integration testing và k6.

Mục đích: lập danh sách chủ đề cần học và câu hỏi nhà tuyển dụng có thể đào sâu từ từng claim trên CV. Tài liệu không có đáp án mẫu.

Nguyên tắc: mọi con số và claim trong CV đều có thể bị hỏi “đo bằng cách nào?”, “trong điều kiện nào?” và “bạn trực tiếp làm phần nào?”. Chỉ giữ lại claim mà bạn có thể giải thích và chứng minh.

MỤC LỤC

- Phần I. Phương pháp luyện trả lời theo khung năm bước
- Phần II. Cách đọc CV dưới góc nhìn interviewer
- Phần III. Đề cương công nghệ — Java, Spring Boot và API
- Phần IV. Đề cương công nghệ — WebSocket/STOMP và real-time collaboration
- Phần V. Đề cương công nghệ — Redis, Kafka và event-driven design
- Phần VI. Đề cương công nghệ — PostgreSQL, JPA/Hibernate và concurrency
- Phần VII. Đề cương công nghệ — Security, JWT và HMAC
- Phần VIII. Đề cương công nghệ — Docker, AWS, CI/CD, testing và performance
- Phần IX. Câu hỏi đào sâu trực tiếp từ hai project
- Phần X. Bài tập cốt lõi và ngân hàng 100 câu hỏi

Nguồn chính: CV người dùng tải lên. Các câu hỏi là dự đoán có cơ sở từ công nghệ và thành tích được ghi trong CV, không phải khẳng định rằng mọi nhà tuyển dụng đều sẽ hỏi tất cả.

PHẦN I — PHƯƠNG PHÁP LUYỆN TRẢ LỜI

Dùng khung năm bước cho mỗi câu hỏi công nghệ. Trước tiên tự trả lời mà không mở tài liệu; sau đó mới kiểm chứng bằng code, log, test hoặc tài liệu chính thức.

1. Definition — Khái niệm là gì?

Định nghĩa chính xác, mục đích và vấn đề được giải quyết. Phân biệt với thuật ngữ gần giống.

2. Mechanism — Nó hoạt động như thế nào?

Trình bày luồng xử lý, thành phần tham gia, dữ liệu đi qua đâu và trạng thái thay đổi như thế nào.

3. Example — Ví dụ thực tế hoặc code minh họa.

Liên hệ trực tiếp với CollabBoard hoặc RushSeat; dùng sequence diagram, code ngắn, SQL, cấu hình hoặc test nếu phù hợp.

4. Limitation — Những trường hợp không hoạt động như mong đợi.

Nêu edge case, failure mode, giới hạn về consistency, ordering, throughput, security, cấu hình hoặc phiên bản.

5. Trade-off — Khi nào nên hoặc không nên sử dụng?

So sánh lựa chọn thay thế, chi phí vận hành, độ phức tạp, latency, durability và khả năng mở rộng.

Riêng với mọi con số trên CV

- Định nghĩa metric rõ ràng: concurrency, throughput, P95/P99 latency, database write load hoặc recovery rate được tính thế nào.
- Trình bày baseline và điều kiện so sánh: cùng workload, dataset, phần cứng, cấu hình, thời gian chạy và số lần lặp hay không.
- Nêu công cụ và bằng chứng: k6 script, dashboard, log, query metrics, benchmark report hoặc commit.
- Phân biệt kết quả đo được với mục tiêu thiết kế; không mô tả số liệu giả lập như production traffic.
- Giải thích phần việc cá nhân, phần do thư viện/framework cung cấp và phần do nhóm thực hiện.

Mẫu ghi chú cho một câu hỏi

Trường	Nội dung cần tự điền
Câu hỏi / chủ đề	Câu hỏi chính xác và project liên quan.
Definition	Định nghĩa ngắn gọn bằng lời của mình.
Mechanism	Các bước xử lý, thành phần, state transition hoặc sequence.
Example	Code, test, sơ đồ, log hoặc ví dụ từ project.
Limitation	Ít nhất một edge case hoặc failure mode.
Trade-off	Phương án thay thế và lý do lựa chọn.
Evidence	File code, test, benchmark, dashboard hoặc tài liệu có thể mở ra.
Mức tự tin	0 = chưa biết; 1 = định nghĩa; 2 = giải thích cơ chế; 3 = áp dụng/debug; 4 = bảo vệ trade-off.

PHẦN II — ĐỌC CV DƯỚI GÓC NHÌN INTERVIEWER

1. CollabBoard — những điểm dễ bị hỏi sâu

- 1.500+ concurrent WebSocket connections và P95 từ 180 ms xuống dưới 42 ms: tải được tạo như thế nào, latency đo ở đâu, mẫu số và thời gian đo là gì.
- Redis Pub/Sub broadcast qua nhiều Spring Boot nodes: vì sao chọn Pub/Sub, điều gì xảy ra khi subscriber mất kết nối hoặc node restart.
- Kafka room-partitioned topics và Redis Hashes: partition key, ordering, consumer concurrency, cache consistency và database write path.
- JWT authentication/authorization cho WebSocket/STOMP frames: handshake, CONNECT/SUBSCRIBE/SEND, token validation và kiểm tra quyền theo room.
- Out-of-order frames và 50 simulated editors: định nghĩa sequence, reconciliation, duplicate/lost operations và cách xác nhận recovery.
- Docker, EC2, RDS, GitHub Actions: deployment path, secrets, migration, health check, rollback và observability.

2. RushSeat — những điểm dễ bị hỏi sâu

- 5.4x throughput và P99 giảm từ 1.250 ms xuống 78 ms: benchmark có cùng workload không, bottleneck ban đầu là gì, có lặp lại được không.
- Redis Lua scripts cho atomic ticket holds: atomicity trong Redis, TTL, key design, race condition và consistency với PostgreSQL.
- PostgreSQL SELECT FOR UPDATE: transaction boundary, lock order, contention, timeout, deadlock và rollback.
- JWT và HMAC-SHA256 webhook: khác biệt authentication/signature verification, canonical payload, timestamp, replay attack và secret rotation.
- Idempotent webhook consumer và Kafka: duplicate delivery, database transaction, offset/ack, retry và dead-letter handling.
- Redis TTL 5 phút: expiration semantics, cleanup timing, reservation state transition và cách tránh giải phóng vé đã confirm.

3. Câu hỏi xác minh ownership

- Bạn trực tiếp thiết kế và implement phần nào? Phần nào được hướng dẫn hoặc dùng thư viện có sẵn?
- Đâu là commit hoặc pull request thể hiện phần việc đó?
- Bạn đã thử nghiệm điều gì trước khi chọn công nghệ này?
- Nếu bỏ Redis/Kafka, hệ thống sẽ đơn giản hơn ở đâu và yếu đi ở đâu?
- Claim nào có số liệu đo được, claim nào là kết quả từ môi trường mô phỏng?
- Bạn sẽ demo hoặc mở file nào để chứng minh mỗi thành tích?

PHẦN III — JAVA, SPRING BOOT VÀ REST API

A. Java 21 và Java Core

- OOP, interface/implementation, composition, dependency inversion và cách áp dụng vào service/domain code.
- Generics, collections, equals/hashCode, immutability và lựa chọn cấu trúc dữ liệu.
- Exception hierarchy, exception translation, validation và cách giữ nguyên cause.
- Concurrency cơ bản: thread safety, shared mutable state, atomicity, visibility, happens-before.
- ExecutorService/CompletableFuture ở mức cần thiết cho background work.
- JVM memory, GC, object allocation và profiling khi xử lý tải.
- Date/time, timezone, BigDecimal, UUID và thiết kế kiểu dữ liệu domain.
- Testing: JUnit, test doubles, parameterized tests và kiểm thử edge case.

B. Spring Boot / Spring Core

- IoC/DI, ApplicationContext, bean registration, component scanning, @Bean và dependency resolution.
- Bean lifecycle, singleton scope, configuration properties, profiles và auto-configuration.
- AOP/proxy: self-invocation, method interception và các tính năng dựa trên proxy.
- Spring MVC request lifecycle, controller/service/repository responsibilities và validation.
- Exception handling bằng @ControllerAdvice, HTTP status, error response contract.
- Configuration management, profiles, secrets và startup failure debugging.
- Async/scheduling/retry: executor, timeout, backoff, idempotency và multi-instance concerns.
- Spring Security filter chain, authentication vs authorization, SecurityContext và method security.

C. REST API và OpenAPI

- HTTP methods, status codes, idempotent methods, safe methods và semantics của POST/PUT/PATCH/DELETE.
- Request validation, DTO, serialization, error model, pagination, filtering và sorting.
- API versioning, backward compatibility, contract-first design và OpenAPI/Swagger.
- Authentication/authorization, CORS, rate limiting và input validation.
- Timeout, retry và lý do không nên retry POST một cách mù quáng.
- API observability: request ID, structured logging, latency metrics và error rate.

Câu hỏi nên tự đặt khi ôn

- Vì sao chọn Spring Boot thay vì tự cấu hình Spring Framework?
- Request đi từ network vào controller rồi đến database theo những bước nào?
- Ở đâu nên đặt validation và business invariants? Tại sao?
- Bạn phân biệt lỗi 400, 401, 403, 404, 409 và 500 trong API như thế nào?
- Làm sao giữ backward compatibility khi thay đổi response DTO?

PHẦN IV — WEBSOCKET/STOMP VÀ REAL-TIME COLLABORATION

A. WebSocket fundamentals

- HTTP request/response so với WebSocket connection; handshake và connection lifecycle.
- Full-duplex communication, frame, ping/pong, close code và disconnect detection.
- Connection state, heartbeat, idle timeout, reconnect và backoff.
- Scaling connection state qua nhiều application nodes; sticky sessions và giới hạn của chúng.
- Backpressure, message size, slow consumer và giới hạn outbound buffer.
- Authentication ở handshake và authorization cho từng message.

B. STOMP và Spring messaging

- STOMP frame: CONNECT, CONNECTED, SUBSCRIBE, SEND, MESSAGE, ACK/NACK và DISCONNECT.
- Destination prefix, application destination, broker destination và user destination.
- Spring message flow: inbound channel, handler, broker channel và outbound channel.
- ChannelInterceptor và cách xác thực/ủy quyền frame.
- Quyền truy cập room/board: ai được subscribe, ai được gửi operation và cách kiểm tra membership.
- Session lifecycle, Principal, user destinations và xử lý reconnect.
- Message ordering, duplicate delivery, sequence number và idempotent operation handling.

C. Collaborative state và synchronization

- Định nghĩa canvas operation, operation ID, board/room ID và sequence/version.
- Optimistic sequencing và client-side reconciliation được mô tả trong CV: thuật toán cụ thể cần tự xác định.
- Out-of-order, duplicate, missing frames, reconnect và resynchronization.
- Conflict resolution: last-write-wins, operation ordering, OT/CRDT ở mức khái niệm và khi nào cần chúng.
- Snapshot, replay log, event history và cách phục hồi state sau khi client reconnect.
- Consistency model giữa client state, Redis cache, Kafka stream và PostgreSQL persistence.
- Đo recovery rate: định nghĩa '100% recovered', số case, loại lỗi và giới hạn của test.

PHẦN V — REDIS, KAFKA VÀ EVENT-DRIVEN DESIGN

A. Redis

- Data structures: String, Hash, Set, Sorted Set, List, Stream và chọn theo access pattern.
- Redis Pub/Sub: delivery semantics, subscriber offline, message loss và phạm vi broadcast.
- Redis Hashes để lưu transient canvas state: key schema, field design, memory footprint và expiration.
- TTL và expiration: thời điểm hết hạn logic, cơ chế dọn key và không xem TTL như scheduler chính xác tuyệt đối.
- Lua scripts: atomic execution trong Redis, KEYS/ARGV, script cache và giới hạn thời gian chạy.
- Atomic ticket hold: kiểm tra tồn kho, tạo hold, decrement/increment và rollback logic trong một script.
- Redis persistence/replication/failover ở mức cần thiết; phân biệt cache với source of truth.
- Eviction policy, maxmemory, hot keys, key cardinality và observability.
- Redis distributed lock: lease, expiry, fencing token và failure assumptions nếu có sử dụng.
- Consistency giữa Redis và PostgreSQL; cache invalidation, compensation và reconciliation.

B. Apache Kafka

- Topic, partition, broker, producer, consumer, consumer group và offset.
- Partition key theo room: đảm bảo ordering trong phạm vi partition và tác động tới phân phối tải.
- Partition count, skew/hot partition, consumer parallelism và throughput.
- Producer acknowledgments, retries, idempotent producer và giới hạn của exactly-once semantics.
- Consumer offset commit, at-most-once, at-least-once và duplicate processing.
- Retry topic, dead-letter topic, poison message và chiến lược xử lý lỗi.
- Consumer lag, rebalance, partition assignment và xử lý khi consumer restart.
- Kafka durability và retention so với Redis Pub/Sub transient messaging.
- Kafka event schema, schema evolution, compatibility và versioning.
- Database update và Kafka offset: dual-write problem, transactional outbox và idempotent consumer.

C. Event-driven architecture

- Phân biệt synchronous HTTP call, in-process event, Redis Pub/Sub và Kafka event.
- Event notification vs event-carried state transfer; mức coupling giữa producer và consumer.
- Event ordering, replay, duplicate, delayed event và eventual consistency.
- Observability xuyên suốt producer-consumer bằng correlation ID.
- Xác định source of truth cho canvas state, reservation state và ticket inventory.
- Thiết kế failure recovery khi Redis, Kafka hoặc PostgreSQL tạm thời không khả dụng.

PHẦN VI — POSTGRESQL, JPA/HIBERNATE VÀ CONCURRENCY

A. PostgreSQL và transaction

- ACID và transaction boundary; commit/rollback.
- MVCC và isolation levels; READ COMMITTED, REPEATABLE READ, SERIALIZABLE.
- Row-level lock, SELECT FOR UPDATE, lock wait, lock timeout và deadlock.
- Lock ordering nhất quán; phát hiện và giảm deadlock.
- Unique constraint, check constraint, foreign key và vai trò của database trong bảo vệ invariants.
- Index, query plan, EXPLAIN ANALYZE và tác động tới latency/write amplification.
- Connection pool, transaction duration, connection exhaustion và timeout.
- Optimistic locking bằng version column so với pessimistic locking.
- Phân biệt khóa trong JVM với database lock khi chạy nhiều instance.

B. JPA/Hibernate

- Entity lifecycle, persistence context, dirty checking và flush.
- Managed, detached, transient và removed entities.
- Lazy/eager loading, N+1 query và cách phát hiện.
- Transaction boundary khi dùng Spring Data JPA.
- OptimisticLockException, pessimistic lock modes và cách kiểm thử.
- Mapping quan hệ, cascade, orphan removal và tác động ngoài ý muốn.
- Batch insert/update, pagination và hiệu năng ORM.
- Native SQL vs JPQL/Criteria; khi nào cần truy vấn trực tiếp.
- Migration schema, backward compatibility và deploy database changes.

C. RushSeat-specific concurrency

- Định nghĩa invariant: một ticket/seat không được cấp cho nhiều reservation đang hợp lệ.
- Atomic Redis hold và PostgreSQL SELECT FOR UPDATE: vai trò từng lớp và cách tránh hai source of truth mâu thuẫn.
- Race giữa reserve, confirm, cancel và expire; liệt kê các interleaving nguy hiểm.
- Lock ordering: tài nguyên nào khóa trước, vì sao thứ tự thống nhất giảm deadlock.
- Reservation state machine: HELD, CONFIRMED, CANCELLED, EXPIRED và các transition hợp lệ.
- TTL hết hạn nhưng database chưa cập nhật; cách reconciliation đảm bảo inventory đúng.
- Retry transaction khi deadlock hoặc serialization failure; giới hạn retry và backoff.
- Benchmark contention: phân biệt throughput tổng, successful booking throughput và error rate.

PHẦN VII — SECURITY: SPRING SECURITY, JWT VÀ HMAC

A. Spring Security và JWT

- Authentication vs authorization; filter chain và SecurityContext.
- JWT structure: header, payload, signature; claims, issuer, audience, expiry và not-before.
- Signature validation, algorithm allowlist, key management và secret rotation.
- Stateless session có nghĩa gì; lợi ích và các giới hạn như revoke token.
- JWT trong HTTP API so với WebSocket handshake/STOMP frames.
- Authorization theo resource/room: xác thực người dùng không tự động chứng minh họ có quyền truy cập mọi room.
- Token expiry, refresh, clock skew, logout/revocation và lưu trữ token an toàn.
- Không đưa secret hoặc token nhạy cảm vào log; kiểm soát CORS/CSRF theo kiến trúc thực tế.

B. HMAC-SHA256 cho payment webhook

- HMAC là gì và khác hash thông thường hoặc chữ ký bất đối xứng thế nào.
- Canonical payload: byte sequence chính xác được ký; ảnh hưởng của JSON serialization và encoding.
- Constant-time comparison và tránh so sánh chữ ký theo cách dễ rò rỉ timing.
- Timestamp, nonce/event ID, replay attack và cửa sổ thời gian chấp nhận.
- Secret storage, rotation, nhiều key đang hiệu lực và quy trình thu hồi.
- Thứ tự xử lý: verify signature, validate event, kiểm tra idempotency, cập nhật trạng thái và phát sự kiện.
- Phản hồi HTTP cho webhook lỗi; lựa chọn retry provider và dead-letter/manual recovery.
- Đo overhead dưới tải: cách tách thời gian verify khỏi network, DB và queue.

C. Threat modeling

- Attacker có thể gửi payload giả, replay request, đoán ID hoặc truy cập room không thuộc quyền.
- Validation và authorization ở server; không tin client-side reconciliation.
- Rate limiting, payload size limits, connection limits và chống resource exhaustion.
- Audit log đủ dùng nhưng không ghi token, secret hoặc thông tin thanh toán nhạy cảm.
- Phân tích failure modes của authentication khi Redis/database không sẵn sàng.

PHẦN VIII — DOCKER, AWS, CI/CD, TESTING VÀ PERFORMANCE

A. Docker và triển khai

- Docker image vs container; Dockerfile layers, build context, caching và multi-stage build.
- Container port, host port, environment variables, secrets và network.
- Health check, readiness/liveness concepts, graceful shutdown và signal handling.
- Container resource limits, JVM memory sizing và OOM behavior.
- Local Docker Compose so với triển khai EC2; cấu hình khác nhau giữa môi trường.
- Image tag/version, reproducible build, vulnerability scanning và least-privilege runtime.

B. AWS EC2/RDS

- EC2 instance, security group, inbound/outbound rules, IAM role và SSH access.
- RDS connectivity, subnet/network access, credentials, TLS, connection limits và backup.
- Phân biệt database public/private access và nguyên tắc least privilege.
- Environment configuration, secrets management và credential rotation.
- Deployment, rollback, migration strategy và downtime considerations.
- Monitoring CPU, memory, network, database connections, query latency và storage.
- Availability/recovery: backup restore, instance failure và giới hạn của một EC2 instance.

C. GitHub Actions CI/CD

- Workflow, event trigger, job, step, runner, artifact và cache.
- Build/test pipeline; phân biệt unit test, integration test và load test.
- Secrets/context, permission scopes, OIDC/IAM ở mức cần thiết và tránh lộ credential.
- Build once, deploy artifact đã kiểm thử, environment approvals và rollback.
- Database migration trong pipeline; thứ tự deploy app/schema và compatibility.
- Failure notification, test report, artifact retention và reproducibility.

D. JUnit, integration testing và k6

- Unit test vs integration test vs end-to-end test; chọn test level cho từng rủi ro.
- Testcontainers hoặc môi trường phụ thuộc thật cho PostgreSQL/Redis/Kafka khi phù hợp.
- Test concurrency: simultaneous requests, barriers, repeated runs và kiểm tra invariant.
- k6 virtual users, arrival-rate executors, ramp-up, duration, thresholds và checks.
- Throughput (req/s), latency distribution, P95/P99, error rate và successful operation rate.
- Closed workload vs open workload; coordinated omission ở mức khái niệm và ảnh hưởng tới kết luận.
- Baseline, warm-up, test duration, hardware, dataset, connection pool và reproducibility.
- Phân biệt benchmark đơn lẻ với load/stress/soak test.
- Quan sát bottleneck: CPU, network, DB lock, Redis, Kafka lag, thread pool và connection pool.
- Chứng minh các claim 1.500 connections, P95 <42 ms, 1.750+ req/s và P99 78 ms bằng artifact nào.

PHẦN IX — CÂU HỎI ĐÀO SÂU TRỰC TIẾP TỪ HAI PROJECT

Phần này giúp bạn kiểm tra các claim nổi bật trên CV. Ưu tiên trả lời bằng bằng chứng thật từ source code, test, script và kết quả benchmark. Không tự tạo số liệu hoặc nhận phần việc mà mình không trực tiếp thực hiện.

A. Java, Spring Boot và API

- Trong hai project, bạn dùng Java 21 ở phần nào và tính năng Java hiện đại nào thực sự giúp ích cho thiết kế?
- Bạn phân chia trách nhiệm controller, service, repository và domain logic như thế nào? Vì sao?
- Spring IoC/DI giải quyết vấn đề gì trong project và dependency được resolve khi startup ra sao?
- Bean singleton có an toàn khi xử lý nhiều WebSocket message đồng thời không? Điều kiện nào quyết định điều đó?
- Bạn xử lý exception và chuyển lỗi nghiệp vụ thành HTTP response như thế nào?
- Bạn thiết kế API reservation để client retry không tạo booking trùng như thế nào?
- Bạn phân biệt unit test, integration test và load test trong pipeline ra sao?
- Bạn debug một lỗi chỉ xuất hiện khi chạy nhiều request đồng thời như thế nào?
- Bạn quản lý cấu hình dev/test/prod và secrets ra sao?
- Bạn sẽ thay đổi điều gì trong kiến trúc hiện tại nếu số lượng người dùng tăng 10 lần?

B. WebSocket, STOMP và CollabBoard

- WebSocket khác HTTP polling và Server-Sent Events như thế nào? Vì sao chọn WebSocket?
- Handshake WebSocket diễn ra như thế nào và khi nào xác thực JWT được thực hiện?
- STOMP cung cấp abstraction gì trên WebSocket? CONNECT, SUBSCRIBE và SEND khác nhau thế nào?
- Spring xử lý một inbound STOMP frame qua những component nào?
- Bạn kiểm tra quyền truy cập board/room ở đâu để tránh user subscribe room trái phép?
- Redis Pub/Sub giúp broadcast giữa các Spring Boot node như thế nào? Subscriber offline thì sao?
- Redis Pub/Sub khác Kafka về durability, replay, ordering và consumer model thế nào?
- Bạn định nghĩa và đo 1.500+ concurrent WebSocket connections như thế nào?
- Bạn đo P95 latency từ điểm nào tới điểm nào? Có bao gồm client processing và network không?
- Bạn chứng minh P95 giảm từ 180 ms xuống dưới 42 ms bằng baseline, workload và artifact nào?
- Bạn xử lý disconnect, reconnect, heartbeat và duplicate subscription như thế nào?
- Bạn xử lý slow consumer hoặc client không đọc message kịp như thế nào?
- Bạn định nghĩa canvas operation ID, sequence number và room ID như thế nào?
- Client-side reconciliation xử lý out-of-order operation theo thuật toán cụ thể nào?
- Claim khôi phục 100% out-of-order frames dựa trên bao nhiêu test case và có giới hạn gì?

C. Redis và atomic ticket holds

- Bạn chọn Redis Hashes để lưu transient canvas state vì lý do gì? Key schema được thiết kế thế nào?
- Redis Pub/Sub và Redis Hashes có vai trò khác nhau thế nào trong CollabBoard?
- Lua script được Redis thực thi atomic theo nghĩa nào? Điều đó không bảo đảm điều gì ở PostgreSQL?
- Bạn thiết kế Lua script để kiểm tra inventory và giữ vé trong một thao tác ra sao?
- Điều gì xảy ra nếu client timeout nhưng Redis đã thực thi script thành công?
- TTL 5 phút hoạt động thế nào và vì sao không nên giả định key sẽ được dọn đúng chính xác tại thời điểm đó?
- Bạn xử lý race giữa TTL expiration và confirmation như thế nào?
- Redis failover hoặc restart có thể ảnh hưởng ticket hold ra sao tùy cấu hình persistence/replication?
- Bạn chọn eviction policy và giới hạn memory như thế nào cho key có TTL?
- Bạn đo throughput tăng từ 320 lên 1.750+ req/s bằng workload và tiêu chí thành công nào?

D. Kafka và event-driven processing

- Bạn chọn room ID làm Kafka partition key vì lý do gì? Điều gì được đảm bảo về ordering?
- Partition count ảnh hưởng thế nào đến parallelism và throughput của consumer group?
- Bạn xử lý hot room hoặc skew partition như thế nào?
- Consumer xử lý event xong nhưng chưa commit offset thì chuyện gì có thể xảy ra?
- At-least-once delivery có thể tạo duplicate như thế nào và consumer phải làm gì?
- Bạn thiết kế idempotent webhook consumer bằng event ID hoặc business key như thế nào?
- Bạn cập nhật database và commit Kafka offset theo thứ tự nào để giảm nguy cơ mất hoặc xử lý lặp?
- Bạn dùng retry topic hoặc dead-letter topic trong trường hợp nào?

- Bạn đo consumer lag và xác định consumer không theo kịp producer như thế nào?
- Bạn giải thích dual-write problem và khi nào cần transactional outbox?

E. PostgreSQL, JPA và concurrency

- SELECT FOR UPDATE khóa những gì và lock được giữ tới thời điểm nào?
- Transaction isolation level của PostgreSQL ảnh hưởng thế nào đến thao tác giữ vé?
- Vì sao hai request có thể cùng đọc thấy còn một vé nếu chỉ kiểm tra trước rồi mới cập nhật?
- Bạn đảm bảo inventory không âm và không oversell bằng invariant/constraint/locking nào?
- Deadlock xảy ra như thế nào trong luồng đặt vé? Consistent lock ordering giảm nguy cơ ra sao?
- Optimistic locking khác pessimistic locking thế nào và benchmark lựa chọn dựa trên tiêu chí nào?
- Bạn chứng minh không oversell bằng test concurrency nào? Kiểm tra invariant sau test ra sao?
- JPA persistence context và dirty checking hoạt động thế nào trong transaction?
- Bạn phát hiện N+1 query hoặc query chậm như thế nào?
- Bạn xử lý retry transaction sau deadlock mà không lặp side effect ngoài database như thế nào?

F. Security, JWT và HMAC

- JWT authentication khác authorization thế nào trong Spring Security?
- JWT được kiểm tra signature, issuer, audience và expiry như thế nào?
- Với WebSocket/STOMP, bạn xác thực kết nối và authorize từng destination/frame ở đâu?
- HMAC-SHA256 khác hash SHA-256 thông thường như thế nào?
- Bạn tạo canonical payload để verify webhook signature thế nào khi JSON có thể khác whitespace/order?
- Vì sao cần constant-time comparison khi so sánh signature?
- Bạn ngăn replay attack bằng timestamp, nonce hoặc event ID như thế nào?
- Bạn xử lý secret rotation và nhiều key hợp lệ trong giai đoạn chuyển đổi ra sao?
- Bạn đảm bảo webhook duplicate không tạo payment side effect lần hai như thế nào?
- Bạn đo validation overhead dưới 5 ms ở 3.000 req/s như thế nào và đã loại trừ những chi phí nào?

G. Docker, AWS, CI/CD và testing/performance

- Bạn đóng gói Spring Boot bằng Docker như thế nào và vì sao chọn cấu trúc Dockerfile đó?
- Ứng dụng EC2 kết nối RDS qua network/security group ra sao? Secret được lưu ở đâu?
- Pipeline GitHub Actions chạy những bước nào trước khi deploy?
- Bạn xử lý database migration tương thích với phiên bản app cũ/mới như thế nào?
- Bạn triển khai rollback nếu deploy mới gây lỗi như thế nào?
- Bạn dùng JUnit cho unit test và integration test ra sao? Test nào cần PostgreSQL/Redis/Kafka thật?
- k6 virtual users khác arrival-rate workload như thế nào?
- P95 và P99 được tính trên tập mẫu nào? Vì sao average latency có thể che giấu vấn đề?
- Bạn phân biệt load test, stress test và soak test như thế nào?
- Bạn xác định bottleneck khi throughput tăng nhưng latency P99 xấu đi bằng metrics và quy trình nào?

H. Ownership, architecture và claim defense

- Vẽ kiến trúc CollabBoard và mô tả đường đi của một canvas operation từ client tới các client khác.
- Vẽ kiến trúc RushSeat và mô tả luồng hold → confirm/cancel/expire.
- Trong mỗi project, thành phần nào là source of truth cho trạng thái nghiệp vụ và vì sao?
- Nếu Redis và PostgreSQL đưa ra trạng thái khác nhau, hệ thống khôi phục consistency thế nào?
- Bạn đã tự implement phần nào, phần nào là framework/library behavior và phần nào là giả lập?
- Trong claim P95 giảm từ 180 ms xuống <42 ms, thay đổi kỹ thuật chính xác nào tạo ra cải thiện và bằng chứng nào hỗ trợ kết luận?
- Trong claim database write load giảm 72%, định nghĩa write load và cách đo trước/sau là gì?
- Trong claim throughput tăng 5.4x, cấu hình baseline và cấu hình mới khác nhau ở những yếu tố nào ngoài code?
- Bạn sẽ demo những file code, test, k6 script, dashboard hoặc workflow nào để chứng minh các claim?
- Nếu làm lại hai project, ba quyết định kỹ thuật nào bạn sẽ giữ, thay đổi hoặc đơn giản hóa? Nêu lý do.

PHẦN X — BÀI TẬP CỐT LÕI

1. Tạo ngân hàng 60–100 câu hỏi phân theo nhóm công nghệ; tài liệu này cung cấp 100 câu hỏi khởi đầu.
2. Mỗi ngày trả lời 5–10 câu không nhìn tài liệu, sử dụng khung Definition → Mechanism → Example → Limitation → Trade-off.
3. Mỗi tuần thực hiện một buổi mock interview 30–60 phút; chọn ngẫu nhiên câu hỏi từ ít nhất ba nhóm công nghệ.
4. Ghi lại câu trả lời sai hoặc thiếu cơ chế; viết lại câu trả lời sau khi kiểm chứng bằng code, test hoặc tài liệu.
5. Với từng bullet thành tích trên CV, chuẩn bị một câu chuyện theo cấu trúc: vấn đề → lựa chọn → triển khai → kiểm thử/đo lường → kết quả → giới hạn.
6. Chuẩn bị bằng chứng cho mọi con số: k6 script, cấu hình test, thời gian chạy, workload, metrics và baseline.
7. Vẽ lại kiến trúc CollabBoard và RushSeat từ trí nhớ; đánh dấu data flow, trust boundary, transaction boundary và failure points.
8. Thực hành một buổi debugging: chọn lỗi concurrency, duplicate event, timeout hoặc stale cache; mô tả cách tái hiện và xác minh bản sửa.
9. Phân loại câu hỏi theo mức tự tin 0–4; ưu tiên ôn những câu dưới mức 3 và các claim nổi bật trên CV.
10. Trước phỏng vấn, rà soát JD và chỉ tăng độ sâu ở công nghệ được yêu cầu; không cần học thuộc toàn bộ mọi công nghệ cùng mức.

Lịch ôn tập gợi ý

Ngày	Nhiệm vụ	Đầu ra
Thứ 2	Java/Spring và kiến thức nền	5–10 câu + một ví dụ code
Thứ 3	Redis/Kafka hoặc WebSocket	Sơ đồ luồng dữ liệu và failure modes
Thứ 4	PostgreSQL/concurrency/security	Một tình huống lỗi và cách kiểm chứng
Thứ 5	Docker/AWS/CI/CD/testing	Mô tả deploy và test pipeline
Thứ 6	Bảo vệ số liệu và claim CV	Bằng chứng cho từng metric
Thứ 7	Mock interview	Danh sách câu sai/thiếu và điểm cần sửa
Chủ nhật	Ôn lại hoặc nghỉ	Cập nhật kế hoạch tuần tiếp theo

Rubric chấm câu trả lời

Điểm	Tiêu chuẩn
0	Không biết hoặc hiểu sai bản chất.
1	Nêu được định nghĩa nhưng không giải thích được cơ chế.
2	Giải thích cơ chế nhưng thiếu ví dụ, bằng chứng hoặc edge case.
3	Có ví dụ cụ thể, nêu được giới hạn và cách kiểm chứng.
4	So sánh trade-off, phân tích failure modes và bảo vệ quyết định bằng bằng chứng.

Checklist trước phỏng vấn

- ☐ Tôi có thể vẽ kiến trúc của cả hai project mà không nhìn tài liệu.
- ☐ Tôi giải thích được từng công nghệ và lý do dùng nó thay vì chỉ đọc tech stack.
- ☐ Tôi biết rõ mỗi metric được định nghĩa, đo và so sánh như thế nào.
- ☐ Tôi phân biệt được điều đã implement với điều framework/library cung cấp.
- ☐ Tôi có thể nêu failure modes của Redis, Kafka, PostgreSQL, WebSocket và deployment.
- ☐ Tôi có thể giải thích security boundary và cách ngăn duplicate/replay.
- ☐ Tôi có bằng chứng thực tế cho các claim; nếu chưa có, tôi điều chỉnh cách trình bày CV cho chính xác.
- ☐ Tôi trả lời được 5–10 câu/ngày và hoàn thành mock interview hàng tuần.

Nguồn tham khảo chính thức

- Java 21 Documentation
- Spring Framework Reference

- Spring Boot Reference
- Spring Security Reference
- Redis Documentation
- Apache Kafka Documentation
- PostgreSQL Documentation
- k6 Documentation
- Docker Documentation
- AWS Documentation
- GitHub Actions Documentation

Tài liệu được xây dựng dựa trên các công nghệ và thành tích nêu trong CV_Backend_InternJunior.pdf. Những câu hỏi yêu cầu chi tiết triển khai cụ thể cần được đối chiếu với source code và bằng chứng thực tế của project.